

De 12% a paridad: qué cubre hoy la versión Laravel

Comparación módulo por módulo entre el sistema anterior en PHP puro y MEGA UNI STORE sobre Laravel 13, después de construir ventas, caja, inventario, compras, reportes y auditoría.

16

TABLAS DE NEGOCIO

16

CONTROLADORES

116

VISTAS

21.428

LÍNEAS DE CÓDIGO

1 Dónde estábamos y dónde estamos

El informe anterior medía 8 de 67 tablas de negocio cubiertas: 12%. Hoy la comparación ya no se puede hacer contando tablas, porque la cobertura dejó de ser proporcional al conteo.

ANTES

12%

8 de 67 tablas

OPERACIÓN DIARIA

100%

vender, cobrar, cuadrar

CONTROL DE EXISTENCIAS

100%

kardex completo

ALCANCE TOTAL DEL VIEJO

62%

falta RR.HH. y contabilidad

La explicación es que las 67 tablas del sistema anterior no pesan lo mismo. Las 9 de ventas y las 2 de inventario son el negocio; las 17 de nómina y contabilidad son *otro* negocio que aquel sistema también resolvía. Lo que se construyó cubre entero el primero.

Cobertura por área

ÁREA	TABLAS VIEJAS	ESTADO HOY	COBERTURA
Ventas y POS	9	Completo	
Inventario y kardex	2	Completo y superado	
Catálogo	8	Completo	
Terceros	4	Completo	
Compras	2	Completo	
Seguridad y auditoría	6	Completo y superado	
Operación (caja, config.)	4	Completo	
Multi-tienda / SaaS	6	Fuera de alcance	
RR.HH. y nómina	17	Sin construir	
Contabilidad	9	Sin construir	



Para una tienda universitaria, el sistema ya sirve. Se puede abrir la caja, vender, cobrar en efectivo o con datáfono, imprimir el recibo, recibir mercancía del proveedor, cuadrar el turno y ver cómo va el mes. Eso es lo que hace un negocio todos los días.

2 Cara a cara

CAPACIDAD	PHP PURO	LARAVEL HOY	DIFERENCIA
Punto de venta	Sí	Sí	Laravel agrega búsqueda por código de barras, atajos de teclado y cobro con varios medios a la vez
Recibo imprimible	Sí	Sí	Formato tirilla con los datos del negocio configurables
Arqueo de caja	Sí	Sí	Desglose por medio de pago y registro de la diferencia
Kardex de inventario	Sí	Sí	Mejor: saldo corriendo, origen del movimiento y bloqueo de fila contra ventas simultáneas
Compras y recepción	Sí	Sí	Borrador y entrada separados; la entrada actualiza costos
Roles y permisos	9 roles	7 roles · 56 permisos	Permiso por acción, no por módulo; el menú se adapta a quien mira
Auditoría	Sí	Sí	Mejor: valor antes y después campo por campo, con IP y navegador
Reportes	Sí	Sí	Siete informes; consultas agregadas en SQL, no filas traídas a PHP
Imágenes de producto	Sí	Sí	Con vista previa antes de subir y borrado del archivo anterior
Multi-tienda	Sí	No	Decisión de alcance: una sola tienda simplifica precio y stock
Nómina	Sí	No	17 tablas de RR.HH. sin construir
Contabilidad	Sí	No	PUC y asientos sin construir
Ventas a crédito	Sí	No	Todo se cobra de contado
Pruebas automáticas	No	3 archivos	Control de acceso, alta de producto y cálculos, contra SQLite en memoria

CAPACIDAD	PHP PURO	LARAVEL HOY	DIFERENCIA
Protección CSRF	Manual	De serie	Laravel la aplica a todos los formularios
Consultas parametrizadas	A mano	Siempre	Eloquent no deja concatenar SQL por accidente

3 Dónde el nuevo es mejor

El inventario no se puede descuadrar por descuido

En el sistema viejo, cualquier consulta podía hacer `UPDATE productos SET stock = ...`. Aquí el saldo solo se mueve por `StockService`, que bloquea la fila del producto, ajusta el saldo y escribe el movimiento de kardex *en la misma transacción*.

```
$fresco = Product::whereKey($producto->getKey())->lockForUpdate()->firstOrFail();
$saldo  =(float) $fresco->stock + $cantidad;
$fresco->forceFill(['stock' => $saldo])->save();
// ...y el movimiento de kardex, dentro de la misma transacción
```

Si dos cajeros venden el último cuaderno al mismo tiempo, uno espera al otro y el segundo recibe el error de existencias. Sin ese bloqueo, los dos venderían y el stock quedaría en -1 .

Las facturas no cambian cuando cambian los precios

Cada renglón de venta guarda copia del nombre, SKU, precio, costo y tarifa de impuesto del momento. Subir el precio de un producto mañana no reescribe la venta de hoy — y como también se guarda el costo, la utilidad histórica es real y no una estimación con los costos de ahora.

Nada desaparece sin dejar rastro

Una venta no se borra: se anula, con motivo y con autor, y la mercancía vuelve al inventario con su propio movimiento. Los productos, categorías, clientes y proveedores usan borrado lógico. Y por encima de todo eso, la auditoría guarda qué campo cambió, de qué valor a qué valor, quién lo hizo, desde qué IP y con qué navegador.

Los permisos son por acción, no por módulo

El sistema viejo daba acceso a «Productos». Aquí un Bodeguero puede ver, crear y editar productos pero **no eliminarlos**; un Vendedor los consulta pero no los toca. Y el menú lateral solo pinta lo que cada quien puede abrir: nadie ve una puerta que le va a dar 403.

4 Lo que falta, en orden de importancia

#	QUÉ	POR QUÉ IMPORTA	ESFUERZO
1	Devoluciones parciales	Hoy solo se anula la venta completa. Si el cliente devuelve un artículo de tres, toca anular todo y volver a vender.	Bajo
2	Ventas a crédito y cartera	Si la tienda fía a estudiantes o a la institución, hoy no hay dónde registrarlo ni cómo cobrarlo.	Medio
3	Valores de atributos y variantes	Existe «Talla» como atributo, pero no «S / M / L» ni la camiseta azul talla M como referencia con su propio stock.	Medio
4	Facturación electrónica DIAN	Obligatoria para vender formalmente en Colombia: resolución, numeración autorizada, CUFE y envío.	Alto
5	Nómina y contabilidad	El sistema viejo las tenía. Antes de construirlas hay que decidir si son parte de este proyecto o de otro.	Muy alto

! **Sobre nómina y contabilidad.** No son «lo que falta para terminar»: son dos sistemas distintos que el proyecto anterior mezcló en la misma base de datos. Construirlos duplicaría el tamaño del proyecto y no haría vender un peso más.

Si el objetivo es una tienda que funcione, ya está. Si el objetivo es replicar el sistema anterior línea por línea, faltan 26 tablas — y vale la pena preguntarse si esas 26 se estaban usando de verdad.

5 Cómo demostrarlo en cinco minutos

Un recorrido que toca todo lo que se construyó, en el orden en que lo haría alguien atendiendo la tienda.

1. **Entra como cajero** (`cajero@megaunistore.test` / `password`) y mira que el menú es más corto que el del administrador: solo lo que le toca.
2. **Caja > Abrir turno** con una base de \$200.000.

3. **Vender:** escribe «cuaderno», toca el producto, sube la cantidad. Prueba a pedir más de lo que hay: el sistema no deja.
4. Pulsa **F9**, usa «pago exacto», confirma. Sale el detalle con la utilidad calculada.
5. **Abre el recibo** y mira los datos del negocio que vienen de Configuración.
6. **Inventario > Kardex** del producto vendido: ahí está la salida con el número de la venta.
7. **Anula la venta** con un motivo. Vuelve al kardex: aparece la entrada de reversión.
8. **Auditoría:** el cambio quedó registrado con tu nombre, la hora y la IP.
9. **Caja > Cerrar:** escribe un valor distinto al esperado y mira cómo calcula el faltante.
10. **Reportes:** cambia el rango a «3 meses» y muestra las gráficas con los 60 días de datos.

En una frase

El sistema pasó de ser un catálogo con ocho pantallas a ser una tienda que funciona: **vende, cobra, controla existencias, compra, cuadra la caja y se puede auditar.**

Frente al proyecto en PHP puro, cubre todo lo que se usa a diario y lo hace con garantías que aquel no tenía — transacciones, bloqueo de filas, permisos por acción, borrado lógico y pruebas automáticas. Lo que queda afuera (nómina, contabilidad, multi-tienda) es una decisión de alcance, no una deuda pendiente.